TREINAMENTO CYPRESS

APOSTILA DO ALUNO · DIA 2

Treinamento Cypress

DIA 01
Fundamentos

DIA 02
Construindo uma suíte

DIA 03
Profissionalizando

```
$ npx cypress run

  Página de login
    ✓ deve carregar a tela de login
    ✓ deve permitir login com credenciais válidas de administrador
    ✓ deve exibir mensagem de erro ao informar senha inválida

  Cadastro de usuário
    ✓ deve cadastrar um usuário comum

  76 passing
```

CONTEÚDO

Sumário

02 DIA 2 — Construindo uma suíte — 3

2.1 Retomada — 3
2.2 Alias — 3
2.3 Testes de API com cy.request — 6
2.4 cy.intercept e login via API — 9
2.5 Testes data driven — 14
2.6 Comandos customizados — 20
2.7 Evidências e plugins — 25
2.8 Lab do Dia 2 — 27
Uma pergunta para levar para casa — 27
Resumo do Dia 2 — 28

APÊNDICES — Consulta rápida — 29

A Comandos do Cypress por finalidade — 29
B Tabela de asserções — 31
C Como ler uma falha: o método de 4 passos — 33
D Mensagens de erro e o que significam — 34
E Seletores do ServeRest — 35
F Comandos customizados do projeto — 37
G Endpoints da API do ServeRest — 38

02

DIA 2

Construindo uma suíte

AO FIM DESTE DIA VOCÊ TERÁ

testes que conversam direto com a API, cenários de erro que você não conseguiria provocar na mão, e seus próprios comandos.

2.1 Retomada

Antes de começar, três perguntas para você responder mentalmente. Se travar em alguma, é a hora de perguntar.

1. Por que `const email = cy.get('...').invoke('val')` não guarda o texto do campo?
2. Por que `cy.wait(3000)` é um problema mesmo quando o teste passa?
3. O que acontece se você rodar só o segundo teste de um arquivo com `it.only`?

2.2 Alias

O problema

Alias é um apelido. Você nomeia alguma coisa uma vez e usa o nome depois. Serve para três coisas diferentes, e a terceira é a mais importante.

Alias de elemento

```javascript
beforeEach(() => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').as('campoEmail')
  cy.get('[data-testid="senha"]').as('campoSenha')
})

it('deve preencher os campos', () => {
  cy.get('@campoEmail').type('admin.treinamento@qa.com')
  cy.get('@campoSenha').type('treino123')
})
```

O `@` diz "isto é um apelido, não um seletor".

Alias de dados

Serve para dados que vêm de uma **fixture**: um arquivo JSON na pasta `cypress/fixtures/`, com dados que o teste lê em vez de tê-los escritos dentro dele. Crie `cypress/fixtures/usuarios.json`:

```json
{
  "admin": {
    "nome": "Administrador do Treinamento",
    "email": "admin.treinamento@qa.com",
    "password": "treino123"
  }
}
```

O `cy.fixture('usuarios')` lê esse arquivo (o `.json` é opcional no nome), e o `.as('usuarios')` dá um apelido ao conteúdo:

```javascript
beforeEach(() => {
  cy.fixture('usuarios').as('usuarios')
})

it('deve logar com administrador', function () {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type(this.usuarios.admin.email)
})
```

ATENÇÃO

A armadilha do `this`. Repare que o `it` acima usa `function () {}`, **não** arrow function. Arrow function não tem `this` próprio — e o alias mora no `this` do teste.

Se você escrever `it('...', () => { ... this.usuarios.admin.email ... })`, recebe: `Cannot read properties of undefined (reading 'usuarios')`. Dentro da arrow function, `this` não é o teste: ele nem existe (`undefined`), e o erro estoura quando você tenta ler o `usuarios` dele.

Esta é a semente que ficou plantada ontem, no bloco 1.2. Agora ela faz sentido, porque existe um problema real na frente.

Se você prefere não pensar em `this`, existe a alternativa:

JAVASCRIPT

```
it('deve logar com administrador', () => {
  cy.get('@usuarios').then((usuarios) => {
    cy.visit('/login')
    cy.get('[data-testid="email"]').type(usuarios.admin.email)
  })
})
```

Funciona igual. Use a que você achar mais legível.

Alias de requisição — o mais importante

JAVASCRIPT

```
cy.intercept('POST', '**/login').as('requisicaoLogin')

cy.get('[data-testid="entrar"]').click()

cy.wait('@requisicaoLogin').its('response.statusCode').should('eq', 200)
```

Aqui o `cy.wait` é legítimo. Olhe a diferença:

	Espera o quê	Problema
`cy.wait(3000)`	Um tempo arbitrário	Chute. Lento se acertou, quebrado se errou
`cy.wait('@login')`	Um acontecimento	Fato. Continua no instante exato em que a resposta chega

DICA

Esta é a ideia central do bloco. Esperar *um tempo* é adivinhação. Esperar *um acontecimento* é engenharia. Sempre que você sentir vontade de escrever um `cy.wait` com número, pergunte: "o que eu estou realmente esperando?" — e espere aquilo.

E você ganha de brinde a capacidade de validar a requisição em si:

```javascript
cy.wait('@requisicaoLogin').then((intercepcao) => {
  expect(intercepcao.response.statusCode).to.eq(200)
  expect(intercepcao.response.body.authorization).to.contain('Bearer')
  expect(intercepcao.request.body.email).to.eq('admin.treinamento@qa.com')
})
```

EXERCÍCIO · 5 MIN

refatore o teste de login do Dia 1 usando alias de elemento.

2.3 Testes de API com `cy.request`

Por que testar API

Porque é **rápido**, **estável** e chega em lugares que a tela não alcança.

	Teste pela tela	Teste pela API
Tempo	Segundos	Milissegundos
Estabilidade	Depende de renderização, animação, rede	Depende só da resposta
Cobertura	Só o que tem botão	Qualquer regra, inclusive as sem tela

Isso **não** substitui o teste de interface — o usuário usa a tela, não a API. Mas boa parte do que a gente testa clicando poderia ser testado muito mais barato.

Leitura simples

```javascript
const apiUrl = Cypress.env('apiUrl')

it('deve listar usuários', () => {
  cy.request('GET', `${apiUrl}/usuarios`).then((res) => {
    expect(res.status).to.eq(200)
    expect(res.body).to.have.property('quantidade')
    expect(res.body.usuarios).to.be.an('array')
  })
})
```

`res` é a resposta inteira: `res.status`, `res.body`, `res.headers`.

Criação

JAVASCRIPT

```javascript
it('deve cadastrar um novo usuário', () => {
  const usuario = {
    nome: 'Usuário Treinamento',
    email: `treino_${Date.now()}@qa.com`,
    password: 'teste123',
    administrador: 'true'
  }

  cy.request('POST', `${apiUrl}/usuarios`, usuario).then((res) => {
    expect(res.status).to.eq(201)
    expect(res.body.message).to.eq('Cadastro realizado com sucesso')
    expect(res.body._id).to.not.be.empty
  })
})
```

O caminho negativo

JAVASCRIPT

```javascript
it('deve impedir cadastro com e-mail duplicado', () => {
  const usuario = {
    nome: 'Usuário Duplicado',
    email: `duplicado_${Date.now()}@qa.com`,
    password: 'teste123',
    administrador: 'false'
  }
  cy.request('POST', `${apiUrl}/usuarios`, usuario)   // cadastra uma vez

  cy.request({                                        // e tenta de novo
    method: 'POST',
    url: `${apiUrl}/usuarios`,
    body: usuario,
    failOnStatusCode: false      // ← sem isto, o teste falha antes de você validar
  }).then((res) => {
    expect(res.status).to.eq(400)
    expect(res.body.message).to.eq('Este email já está sendo usado')
  })
})
```

ATENÇÃO

O e-mail repetido é do próprio teste — o do admin, nunca. É a mesma regra do bloco 1.5, agora pela API: se o admin tiver sumido do ambiente, um `POST` com o e-mail dele o recriaria com outros dados.

> **DICA**
>
> `failOnStatusCode: false`. Por padrão o Cypress considera qualquer status de erro como falha do teste — o que faz sentido, porque geralmente é. Para testar de propósito o caminho que dá errado, você desliga esse comportamento.

Autenticação encadeada

Algumas operações exigem token. Você pede o token e usa na requisição seguinte:

JAVASCRIPT

```
it('deve cadastrar produto autenticado', () => {
  cy.request('POST', `${apiUrl}/login`, {
    email: 'admin.treinamento@qa.com',
    password: 'treino123'
  }).then((login) => {
    const token = login.body.authorization

    cy.request({
      method: 'POST',
      url: `${apiUrl}/produtos`,
      headers: { authorization: token },
      body: {
        nome: `Produto ${Date.now()}`,
        preco: 100,
        descricao: 'Produto de treinamento',
        quantidade: 10
      }
    }).then((res) => {
      expect(res.status).to.eq(201)
    })
  })
})
```

Repare na estrutura: o segundo `cy.request` está **dentro** do `.then()` do primeiro. Ele só pode existir depois que o token chegou.

> **DICA**
>
> Isso fica repetitivo rápido. No bloco 2.6 a gente empacota tudo isso num comando só.

EXERCÍCIO · 15 MIN

ciclo de vida de um usuário pela API — criar, consultar pelo `_id`, excluir, e confirmar que consultar o excluído devolve 400. (Listar e tentar criar duplicado você já viu acima.)

Resolução — confira depois de tentar:

```javascript
it('deve criar, consultar e excluir um usuário', () => {
  const usuario = {
    nome: 'Usuário Descartável',
    email: `descartavel_${Date.now()}@qa.com`,
    password: 'teste123',
    administrador: 'false'
  }

  cy.request('POST', `${apiUrl}/usuarios`, usuario).then((criado) => {
    const id = criado.body._id

    cy.request('GET', `${apiUrl}/usuarios/${id}`).its('body.email').should('eq', usuario.email)

    cy.request('DELETE', `${apiUrl}/usuarios/${id}`).its('body.message').should('eq', 'Registro excluído com sucesso')

    cy.request({ method: 'GET', url: `${apiUrl}/usuarios/${id}`, failOnStatusCode: false })
      .its('status')
      .should('eq', 400)
  })
})
```

O `.its('body.email')` é um atalho: pega uma propriedade da resposta e deixa o `.should()` conferir. É o mesmo que um `.then()` com `expect`, em uma linha.

2.4 `cy.intercept` e login via API

`cy.intercept` observa e, se você quiser, **modifica** o que passa pela rede. Tem três usos, em ordem crescente de poder: **observar**, **substituir** e **forçar o impossível**.

Os três vão olhar a listagem de produtos do admin — e ela só abre para quem está logado. Então, antes deles, uma ferramenta: entrar sem passar pela tela.

Login via API — o padrão que muda sua suíte

Você não precisa testar o login em todos os testes. Precisa testá-lo **uma vez**. Nos outros, entre pela porta dos fundos: peça o token à API e grave-o onde a loja procura.

Em `cypress/e2e/dia2/08-intercept.cy.js`:

JAVASCRIPT

```javascript
const apiUrl = Cypress.env('apiUrl')

// A porta dos fundos: pede o token à API e grava onde o front procura.
// O cy.visit('/login') antes de gravar é o que define ONDE o localStorage fica.
function entrarViaApi() {
  cy.request('POST', `${apiUrl}/login`, {
    email: 'admin.treinamento@qa.com',
    password: 'treino123'
  }).then((res) => {
    cy.visit('/login')
    cy.window().then((win) => {
      win.localStorage.setItem('serverest/userToken', res.body.authorization)
      win.localStorage.setItem('serverest/userNome', 'Administrador do Treinamento')
    })
  })
}

describe('Login via API', () => {
  it('deve entrar sem passar pela tela de login', () => {
    entrarViaApi()
    cy.visit('/admin/home')
    cy.contains('h1', 'Bem Vindo').should('be.visible')
  })

  it('deve voltar para o login quando não há token', () => {
    cy.visit('/admin/home')
    cy.url().should('include', '/login')
  })
})
```

O `serverest/userToken` é o que abre a porta; o `serverest/userNome` é o que a tela do admin imprime em `Bem Vindo <nome>`.

> **CRÍTICO**
>
> **O `cy.visit('/login')` antes não é desperdício.** É ele que estabelece a origem onde o `localStorage` vai ser gravado. Se você escrever antes de qualquer visita, grava no lugar errado — e o teste volta para a tela de login sem dizer por quê. Esse erro custa muito tempo de quem não sabe.

Por que isso importa: uma suíte de trinta testes que faz login pela tela gasta uns três segundos por teste só nisso. São **um minuto e meio** de execução, e trinta oportunidades de o teste falhar por um motivo que não tem nada a ver com o que ele deveria validar.

> **DICA**
>
> Por enquanto a função mora dentro deste arquivo. No bloco 2.6 ela vira o comando `cy.loginViaApi()`, disponível para todos os testes.

Uso 1 — espiar e esperar

JAVASCRIPT

```
it('deve esperar a listagem de produtos carregar', () => {
  cy.intercept('GET', '**/produtos*').as('listagemProdutos')

  entrarViaApi()                        // a listagem é do admin: sem sessão, volta ao /login
  cy.visit('/admin/listarprodutos')

  cy.wait('@listagemProdutos').its('response.statusCode').should('eq', 200)
  cy.contains('h1', 'Lista dos Produtos').should('be.visible')
})
```

CRÍTICO

Registre o intercept ANTES do `cy.visit`. Ele só captura o que passar depois de existir. Registrado depois, não pega nada.

DICA

Repare no `*` no fim de `**/produtos*`. Ele faz o padrão casar também com a rota que tem query string (`/produtos?nome=x`). A listagem do ServeRest chama `/produtos` puro, então aqui funcionaria sem ele — mas em outra aplicação, com filtro ou paginação na URL, é o que evita um `cy.wait` que estoura sem explicar por quê. Quando o intercept não casar, abra a aba Network do DevTools e compare com a URL real.

Uso 2 — simular a resposta

Em vez de deixar a requisição ir ao servidor, você responde por ele. Crie `cypress/fixtures/produtos-mock.json` com dois produtos inventados:

JSON

```json
{
  "quantidade": 2,
  "produtos": [
    {
      "nome": "Produto Simulado A",
      "preco": 10,
      "descricao": "Resposta simulada — não existe no servidor",
      "quantidade": 1,
      "_id": "mock0000000000A"
    },
    {
      "nome": "Produto Simulado B",
      "preco": 20,
      "descricao": "Resposta simulada — não existe no servidor",
      "quantidade": 2,
      "_id": "mock0000000000B"
    }
  ]
}
```

JAVASCRIPT

```javascript
describe('Uso 2 — simular a resposta', () => {
  beforeEach(() => {
    entrarViaApi()
  })

  it('deve exibir apenas os produtos da fixture', () => {
    cy.intercept('GET', '**/produtos*', { fixture: 'produtos-mock.json' }).as('produtosMock')

    cy.visit('/admin/listarprodutos')
    cy.wait('@produtosMock')

    cy.get('table.table-striped tbody tr').should('have.length', 2)
    cy.contains('td', 'Produto Simulado A').should('be.visible')
  })
})
```

O ganho é **determinismo**. O teste passa a não depender de quantos produtos existem na base naquele momento — que só cresce a cada execução da suíte.

Lista vazia — o cenário que todo mundo esquece e que quebra em produção:

```javascript
it('deve exibir a tabela vazia quando não há produtos', () => {
  cy.intercept('GET', '**/produtos*', { body: { quantidade: 0, produtos: [] } }).as('semProdutos')

  cy.visit('/admin/listarprodutos')
  cy.wait('@semProdutos')

  cy.get('table.table-striped tbody tr').should('have.length', 0)
})
```

DICA

Um achado de graça. Rodando esse teste você descobre que a listagem do ServeRest **não avisa nada** quando está vazia — mostra uma tabela em branco, sem uma linha de texto. Isso é um problema de usabilidade real, e quem encontrou foi o teste.

Guarde esse exemplo: automação não serve só para confirmar o que você já sabe.

Uso 3 — forçar o impossível

Como você testaria a tela de erro 500 sem derrubar o servidor?

```javascript
cy.intercept('GET', '**/produtos*', { statusCode: 500, body: {} }).as('erroServidor')
```

Pronto. O servidor está ótimo; a sua aplicação acha que ele caiu.

E aqui acontece uma coisa interessante com o ServeRest. Rode esse teste e você vai ver a **aplicação quebrar** — não aparecer uma mensagem de erro bonita: quebrar mesmo, com exceção de JavaScript. O teste fica vermelho com `Cannot read properties of undefined (reading 'data')` — e esse erro não é do teste, é da aplicação.

O motivo está no código do ServeRest: ele faz `.catch(e => console.error(e))` e segue adiante tentando ler os dados de uma resposta que não veio.

Você acabou de encontrar um defeito real. Agora vem a decisão interessante, e ela é sobre postura profissional, não sobre Cypress:

```javascript
// ❌ Esconder o problema — o teste passa e o bug some
Cypress.on('uncaught:exception', () => false)
```

JAVASCRIPT

```javascript
// ✅ Transformar o problema na asserção do teste
it('deve expor que a tela não trata o erro 500 do servidor', () => {
  cy.intercept('GET', '**/produtos*', { statusCode: 500, body: {} }).as('erroServidor')

  const erros = []
  cy.on('uncaught:exception', (err) => {
    erros.push(err.message)
    return false
  })

  entrarViaApi()
  cy.visit('/admin/listarprodutos')
  cy.wait('@erroServidor').its('response.statusCode').should('eq', 500)

  // should, não then: a exceção estoura um instante depois da resposta.
  // O should repete a conferência até ela aparecer.
  cy.wrap(erros).should((lista) => {
    expect(lista.join(' '), 'a tela quebra em vez de mostrar uma mensagem').to.contain('undefined')
  })
})
```

O `cy.on('uncaught:exception', ...)` é chamado a cada erro da aplicação. Aqui ele **anota** a mensagem e devolve `false` ("não derrube o teste por isso"); no fim, o teste **afirma** que a quebra aconteceu.

DICA

As duas versões deixam o teste verde. Só uma delas conta a verdade. Um `Cypress.on('uncaught:exception', () => false)` global no arquivo de suporte faz todos os erros da aplicação sumirem — inclusive os que você ainda não descobriu. É silenciar o alarme de incêndio porque ele apita.

ATENÇÃO

Por que `should` e não `then`? A primeira versão deste teste usava `cy.then(...)`. Passava sempre no computador do instrutor e falhou duas vezes em três no servidor de CI, que é mais lento. O `cy.wait` termina quando a resposta chega; a exceção só estoura um instante depois. O `then` confere uma vez e desiste; o `should` tenta de novo até dar certo. Você vai rever isso no Dia 3, na parte de testes instáveis.

2.5 Testes data driven

O problema

Você tem cinco cenários de login inválido. Copia e cola o teste cinco vezes, mudando o e-mail e a mensagem. Agora, quando o seletor do botão mudar, você abre cinco testes.

Separando dados de lógica

`cypress/fixtures/login-invalido.json`:

JSON

```
[
  {
    "cenario": "o e-mail não está cadastrado",
    "email": "naoexiste@qa.com",
    "senha": "treino123",
    "mensagemEsperada": "Email e/ou senha inválidos"
  },
  {
    "cenario": "a senha está incorreta",
    "email": "admin.treinamento@qa.com",
    "senha": "senhaerrada",
    "mensagemEsperada": "Email e/ou senha inválidos"
  },
  {
    "cenario": "o e-mail está em branco",
    "email": "",
    "senha": "treino123",
    "mensagemEsperada": "Email é obrigatório"
  },
  {
    "cenario": "a senha está em branco",
    "email": "admin.treinamento@qa.com",
    "senha": "",
    "mensagemEsperada": "Password é obrigatório"
  }
]
```

Repare que o `cenario` foi escrito para completar uma frase: ele vai virar o nome do teste — *"deve exibir erro quando o e-mail não está cadastrado"*.

O teste, uma vez só:

JAVASCRIPT

```
const cenarios = require('../../fixtures/login-invalido.json')

describe('Login — cenários inválidos', () => {
  cenarios.forEach((caso) => {
    it(`deve exibir erro quando ${caso.cenario}`, () => {
      cy.visit('/login')

      if (caso.email) cy.get('[data-testid="email"]').type(caso.email)
      if (caso.senha) cy.get('[data-testid="senha"]').type(caso.senha)
      cy.get('[data-testid="entrar"]').click()

      cy.get('.alert').should('be.visible').and('contain', caso.mensagemEsperada)
    })
  })
})
```

Cada item do JSON vira um `it` independente, com nome próprio no relatório. **Adicionar um cenário passou a ser adicionar uma linha no JSON.**

> DICA
> **E o `if`? Não é antipadrão?** Aqui não. O `cy.type()` recusa texto vazio, e o cenário "campo em branco" precisa existir — então o `if` decide **o que preencher**, não **o que validar**. A asserção é sempre a mesma. Você vai ver a diferença com calma no bloco 3.3.

> ATENÇÃO
> **`require` ou `cy.fixture()`?** Para gerar testes em loop, tem que ser `require`.
>
> O `require` acontece quando o arquivo é carregado — antes de qualquer teste rodar. O `cy.fixture()` é um comando, e comando só roda **dentro** de um teste. Se você tentar `cy.fixture().then(dados => dados.forEach(...))` direto no `describe`, o Cypress recusa com uma mensagem que já diz tudo: `Cannot call cy.fixture() outside a running test`.
>
> Regra prática: **`require` para gerar testes, `cy.fixture()` para usar dados dentro de um teste.**

Um detalhe que rende uma boa pergunta

A mensagem para campo vazio depende de **como** ele chega vazio:

O que a requisição manda	A API responde
Não manda a chave `email`	`email é obrigatório`
Manda `email: ""`	`email não pode ficar em branco`

Pela tela, sempre cai na primeira — porque o React não inclui no corpo um campo que nunca foi tocado.

Qual das duas o teste deve esperar? A que o usuário vê. Essa é a regra que vale para o resto da sua carreira: o teste valida o comportamento observável, não a implementação.

> DICA
> Repare também na maiúscula: a API respondeu `email`, a tela mostra `Email`. O front capitaliza antes de exibir. Copiar a mensagem da tela, não da documentação, evita meia hora de confusão.

Segredos e variáveis de ambiente

Até aqui, o e-mail e a senha do admin estavam escritos dentro dos testes. O lugar deles é a configuração. No `cypress.config.js`, o bloco `env` ganha duas linhas:

```javascript
  env: {
    apiUrl: 'http://localhost:3000',
    // Credencial de aplicação de estudo pode ficar aqui. Credencial real, nunca.
    usuarioAdmin: 'admin.treinamento@qa.com',
    senhaAdmin: 'treino123'
  },
```

Salve: o Cypress reinicia sozinho. A partir daí, qualquer teste lê os valores com `Cypress.env()`:

```javascript
describe('Variáveis de ambiente', () => {
  it('deve ler as credenciais da configuração, não do código', () => {
    const email = Cypress.env('usuarioAdmin')
    const senha = Cypress.env('senhaAdmin')

    cy.request('POST', `${Cypress.env('apiUrl')}/login`, { email, password: senha })
      .its('status')
      .should('eq', 200)
  })
})
```

Pela tela, o uso é o mesmo:

```javascript
cy.get('[data-testid="email"]').type(Cypress.env('usuarioAdmin'))
```

E um valor pode ser trocado na hora de rodar, sem mexer em arquivo nenhum:

```terminal
npx cypress run --env usuarioAdmin=outro@qa.com
```

CRÍTICO

Senha nunca vai para o repositório. Nem em fixture, nem em constante, nem "só por enquanto". No servidor de integração, a senha vem de uma variável configurada lá — e quem tem acesso ao código não tem acesso à senha.

Credencial de aplicação de estudo, como o `admin.treinamento@qa.com` deste treinamento, é exceção: não há o que proteger.

DICA

E o aviso amarelo sobre `allowCypressEnv` ? No Cypress 15, o `Cypress.env()` foi depreciado: continua funcionando, mas avisa que vai sumir. **E sumiu: o Cypress 16 (setembro de 2026) o removeu** — por isso você instalou o 15.21.1 no bloco 1.3. Os substitutos:

Para	Use
Valor público, como a URL da API	`Cypress.expose('apiUrl')` , com a chave `expose` no config
Segredo, como a senha	`cy.env(['senhaAdmin'])` — é um comando: o valor chega no `.then()`

Este material continua com `Cypress.env()` porque é o que você ainda vai encontrar em quase toda documentação e projeto, e porque é mais simples de ler — o `cy.env()` exige `.then()` em todo lugar. Num projeto novo, no Cypress 16, use os substitutos desde o início. O conceito é o mesmo: configuração fora do código, segredo fora do repositório. Quando migrar, o valor público muda só de nome; o segredo passa a chegar dentro de um `.then()` , como qualquer comando.

EXERCÍCIO · 5 MIN

crie uma fixture com três produtos e cadastre todos em loop, via API.

ATENÇÃO

O ServeRest recusa dois produtos com o mesmo nome. A fixture guarda o **modelo** do produto; o teste acrescenta um sufixo único. Sem isso, funciona na primeira rodada e falha na segunda.

Resolução — confira depois de tentar. A fixture `cypress/fixtures/produtos.json` :

JSON

```
[
  {
    "nome": "Teclado Mecânico de Treinamento",
    "preco": 250,
    "descricao": "Teclado usado como massa de teste",
    "quantidade": 15
  },
  {
    "nome": "Monitor de Treinamento",
    "preco": 1200,
    "descricao": "Monitor usado como massa de teste",
    "quantidade": 8
  },
  {
    "nome": "Cadeira de Treinamento",
    "preco": 890,
    "descricao": "Cadeira usada como massa de teste",
    "quantidade": 3
  }
]
```

E o teste, no `09-data-driven.cy.js`:

JAVASCRIPT

```
describe('Cadastro de produtos em lote', () => {
  const produtos = require('../../fixtures/produtos.json')
  // A fixture guarda o MODELO. O sufixo torna o nome único: o ServeRest recusa nome repetido.
  const sufixo = Date.now()

  produtos.forEach((produto) => {
    it(`deve cadastrar via API o produto "${produto.nome}"`, () => {
      cy.request('POST', `${Cypress.env('apiUrl')}/login`, {
        email: Cypress.env('usuarioAdmin'),
        password: Cypress.env('senhaAdmin')
      }).then((login) => {
        cy.request({
          method: 'POST',
          url: `${Cypress.env('apiUrl')}/produtos`,
          headers: { authorization: login.body.authorization },
          body: { ...produto, nome: `${produto.nome} ${sufixo}` }
        })
          .its('status')
          .should('eq', 201)
      })
    })
  })
})
```

Repare no `{ ...produto, nome: ... }`: é o spread do bloco 1.2 — copia o produto da fixture e troca só o nome.

2.6 Comandos customizados

O problema

O mesmo bloco de seis linhas aparece em doze arquivos. Quando ele precisa mudar, você abre doze arquivos.

Comandos simples

Em `cypress/support/commands.js` — o arquivo que o Cypress criou no primeiro dia. Tudo o que você registra ali vale para todos os testes:

JAVASCRIPT

```
// Bloco 2.6 — comandos que interagem com a tela.

Cypress.Commands.add('login', (email, senha) => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type(email)
  cy.get('[data-testid="senha"]').type(senha)
  cy.get('[data-testid="entrar"]').click()
})
```

Uso: `cy.login('admin.treinamento@qa.com', 'treino123')`

A porta dos fundos, para todos

A função `entrarViaApi` do bloco 2.4 vira comando, no mesmo arquivo. Duas melhorias: sem argumento, ela usa o admin da configuração (bloco 2.5); e o nome do usuário vem da própria API, em vez de ficar fixo.

```javascript
// A porta dos fundos do bloco 2.4, agora disponível para todos os testes.
Cypress.Commands.add('loginViaApi', (email, senha) => {
  email = email || Cypress.env('usuarioAdmin')
  senha = senha || Cypress.env('senhaAdmin')

  cy.request('POST', `${Cypress.env('apiUrl')}/login`, { email, password: senha }).then((res) => {
    cy.visit('/login')
    cy.window().then((win) => {
      win.localStorage.setItem('serverest/userToken', res.body.authorization)
    })
  })

  cy.request('GET', `${Cypress.env('apiUrl')}/usuarios?email=${email}`).then((res) => {
    cy.window().then((win) => {
      win.localStorage.setItem('serverest/userNome', res.body.usuarios[0].nome)
    })
  })
})
```

Uso: `cy.loginViaApi()` para o admin, ou `cy.loginViaApi(email, senha)` para outro usuário.

Comando filho

Um comando que age sobre o que veio antes:

```javascript
// Comando filho: continua a partir do elemento anterior.
// Uso: cy.get('h1').validarTexto('Bem Vindo')
Cypress.Commands.add('validarTexto', { prevSubject: true }, (elemento, texto) => {
  cy.wrap(elemento).should('contain', texto)
})
```

O `prevSubject: true` é o que diz "eu recebo o elemento anterior".

Os três em uso, em `cypress/e2e/dia2/10-comandos-customizados.cy.js`:

```javascript
describe('Comandos de interface', () => {
  it('deve usar o comando login', () => {
    cy.login('admin.treinamento@qa.com', 'treino123')
    cy.url().should('include', '/admin/home')
  })

  it('deve usar o comando filho validarTexto', () => {
    cy.loginViaApi()
    cy.visit('/admin/home')
    cy.get('h1').validarTexto('Bem Vindo')
  })
})
```

Comandos de API, em arquivo próprio

Os comandos que só falam com a API — preparam dados, não mexem na tela — ficam num segundo arquivo, `cypress/support/api-commands.js`. Naturezas diferentes, arquivos diferentes: fica óbvio onde um comando novo deve morar.

JAVASCRIPT

```
// Bloco 2.6 — comandos que falam com a API. Preparam estado; não mexem na tela.

const apiUrl = () => Cypress.env('apiUrl')

Cypress.Commands.add('obterToken', (email, senha) => {
  return cy
    .request('POST', `${apiUrl()}/login`, {
      email: email || Cypress.env('usuarioAdmin'),
      password: senha || Cypress.env('senhaAdmin')
    })
    .its('body.authorization')
})

// O `||` é o que torna o comando reutilizável: funciona sem argumento nenhum
// e aceita trocar só o que importa para o teste.
Cypress.Commands.add('criarUsuarioViaApi', (dados = {}) => {
  const usuario = {
    nome: dados.nome || 'Usuário Treinamento',
    email: dados.email || `user_${Date.now()}@qa.com`,
    password: dados.password || 'teste123',
    administrador: dados.administrador || 'true'
  }

  return cy
    .request('POST', `${apiUrl()}/usuarios`, usuario)
    .then((res) => ({ ...usuario, _id: res.body._id }))
})
```

O Cypress carrega sozinho o `cypress/support/e2e.js` antes de cada arquivo de teste, e é ele que importa o `commands.js`. Para o arquivo novo valer, importe-o ali também, logo abaixo do import que já existe:

JAVASCRIPT

```
import './commands'
import './api-commands'
```

O padrão que torna um comando realmente reutilizável

Repare no `dados.nome || 'Usuário Treinamento'`: *"use o que me deram; se não me deram nada, use este"*.

O resultado é um comando que atende os dois extremos:

```javascript
cy.criarUsuarioViaApi()                              // não me importa quem, só quero um usuário
cy.criarUsuarioViaApi({ nome: 'Nome Específico' })   // me importa o nome, o resto tanto faz
```

> **DICA**
>
> **É isso que separa um comando útil de um comando que ninguém usa.** Se ele exigir sete parâmetros, as pessoas vão escrever o `cy.request` na mão.

Note também o `return` e o que ele devolve: o objeto do usuário **com o `_id`**. Assim o teste recebe de volta o que precisa — e pode, por exemplo, entrar com o usuário que acabou de criar:

```javascript
describe('Comandos de API', () => {
  it('deve criar usuário sem informar nenhum dado', () => {
    cy.criarUsuarioViaApi().then((usuario) => {
      expect(usuario._id).to.not.be.empty
    })
  })

  it('deve criar usuário trocando só o nome', () => {
    cy.criarUsuarioViaApi({ nome: 'Nome Específico' }).then((usuario) => {
      expect(usuario.nome).to.eq('Nome Específico')
      expect(usuario.password).to.eq('teste123')
    })
  })

  it('deve criar um usuário e entrar com ele', () => {
    cy.criarUsuarioViaApi().then((usuario) => {
      cy.loginViaApi(usuario.email, usuario.password)
      cy.visit('/admin/home')
      cy.contains('h1', usuario.nome).should('be.visible')
    })
  })
})
```

Onde está o limite

Comando customizado é para **ação transversal**: login, criar massa, esperar carregamento. Coisas que servem em qualquer tela.

Se você começar a criar um comando por tela — `cy.preencherFormularioDeProduto`, `cy.validarTabelaDeUsuarios`, `cy.clicarNoBotaoDeCadastroDeUsuario` — o `commands.js` vira um depósito de duzentas linhas e ninguém acha nada.

> **DICA**
>
> **Guarde esse incômodo.** Você provavelmente vai senti-lo no lab de hoje. Amanhã a gente resolve.

EXERCÍCIO · 5 MIN

crie o comando `cadastrarProdutoViaApi`, que já resolva a autenticação por dentro. Pense: por que é melhor ele buscar o token sozinho do que receber um token como parâmetro?

Resolução — confira depois de tentar. No `api-commands.js`:

JAVASCRIPT

```
// Exercício do bloco 2.6: o teste não precisa saber que existe token.
Cypress.Commands.add('cadastrarProdutoViaApi', (dados = {}) => {
  const produto = {
    nome: dados.nome || `Produto ${Date.now()}`,
    preco: dados.preco || 100,
    descricao: dados.descricao || 'Produto de treinamento',
    quantidade: dados.quantidade || 10
  }

  return cy.obterToken(dados.email, dados.senha).then((token) => {
    return cy
      .request({
        method: 'POST',
        url: `${apiUrl()}/produtos`,
        headers: { authorization: token },
        body: produto
      })
      .then((res) => ({ ...produto, _id: res.body._id }))
  })
})
```

E um teste que o usa, no `describe('Comandos de API')`:

JAVASCRIPT

```
it('deve cadastrar produto via API', () => {
  cy.cadastrarProdutoViaApi({ nome: `Produto Comando ${Date.now()}` }).then((produto) => {
    cy.loginViaApi()
    cy.visit('/admin/listarprodutos')
    cy.contains('td', produto.nome).should('be.visible')
  })
})
```

A resposta à pergunta: quem chama o comando não precisa saber que existe token. Se amanhã a autenticação mudar, muda um lugar só — o comando.

2.7 Evidências e plugins

Vídeos e screenshots

JAVASCRIPT

```
{
  video: true,
  videoCompression: 32,
  screenshotOnRunFailure: true,
  trashAssetsBeforeRuns: true
}
```

E manualmente, quando você quer registrar um momento específico:

JAVASCRIPT

```
cy.screenshot('nome-da-evidencia')
cy.get('[data-testid="entrar"]').screenshot('botao-entrar')   // só o elemento
```

Três coisas que vale saber:

- **Vídeo só é gerado em `cypress run`**, não no modo interativo. E vem desligado por padrão desde o Cypress 13: sem `video: true`, não há vídeo
- **Vídeo pesa e desacelera.** Muita equipe guarda só o das falhas
- **Screenshot de falha é automático.** Você não precisa fazer nada

Plugins: o que são e por que existem

Plugin é código Node que estende o Cypress.

Por que eles precisam existir: o seu teste roda **dentro do navegador**. E o navegador, por segurança, não tem acesso ao sistema de arquivos, ao banco de dados nem ao terminal. Plugin é a ponte para fora.

> **DICA**
> Entender esse *porquê* vale mais que decorar uma lista. Quando você precisar de algo que o navegador não alcança — ler um PDF baixado, consultar o banco, rodar um comando — a resposta é "procurar um plugin".

Instalando um: faker

TERMINAL

```
npm install --save-dev @faker-js/faker@10.6.0
```

O `@10.6.0` fixa a versão, pelo mesmo motivo do Cypress no bloco 1.3: é a versão em que o material foi testado.

Depois, no topo do `10-comandos-customizados.cy.js`, o import, e um teste:

JAVASCRIPT

```javascript
import { faker } from '@faker-js/faker/locale/pt_BR'

describe('Massa realista com faker', () => {
  it('deve cadastrar usuário com dados gerados', () => {
    const usuario = {
      nome: faker.person.fullName(),
      // Faker gera dado REALISTA, não ÚNICO. O Date.now() garante a unicidade.
      email: `${faker.internet.username().toLowerCase()}_${Date.now()}@qa.com`,
      password: faker.internet.password({ length: 10 }),
      administrador: 'false'
    }

    cy.criarUsuarioViaApi(usuario).then((criado) => {
      cy.request('GET', `${Cypress.env('apiUrl')}/usuarios/${criado._id}`)
        .its('body.nome')
        .should('eq', usuario.nome)
    })
  })
})
```

Rode duas vezes: cada execução cadastra uma pessoa diferente, com nome brasileiro — é o `locale/pt_BR` no import.

> **ATENÇÃO**
> Repare no `Date.now()` no e-mail, mesmo usando faker. O faker gera dados **realistas**, não **únicos** — ele pode repetir. Realismo não substitui unicidade num ambiente compartilhado.

Plugins que vale conhecer

Plugin	Para quê
`cypress-mochawesome-reporter`	Relatório HTML
`@faker-js/faker`	Massa realista
`cypress-real-events`	Eventos reais de mouse e teclado (hover, por exemplo)
`cypress-plugin-api`	Visualização rica de requisições
`cypress-axe`	Testes de acessibilidade
`@badeball/cypress-cucumber-preprocessor`	Gherkin / BDD

> ATENÇÃO
> **Upload de arquivo não precisa mais de plugin.** Tutoriais antigos indicam o `cypress-file-upload`, que foi descontinuado: desde o Cypress 9.3 o comando nativo `.selectFile('caminho/do/arquivo')` resolve.

2.8 Lab do Dia 2

40 minutos. Crie o arquivo `cypress/e2e/dia2/lab-dia2.cy.js`.

Parte 1 — API

- Criar um usuário administrador usando comando customizado
- Autenticar e cadastrar um produto
- Validar o produto na listagem da API

Parte 2 — Interface

- Entrar usando login via API
- Validar que o produto aparece na tela
- Interceptar a requisição de listagem e validar o status

Parte 3 — Data driven

- Fixture com ao menos três cenários de login inválido (a `login-invalido.json` do projeto já tem quatro; pode usá-la)
- Gerar os testes em loop

Requisitos técnicos: ao menos dois comandos customizados, um alias de requisição, nenhum `cy.wait` numérico, massa criada pelo próprio teste.

Comandos que você criou no bloco 2.6:

```javascript
cy.criarUsuarioViaApi({ nome, email, password, administrador })
cy.cadastrarProdutoViaApi({ nome, preco, descricao, quantidade })
cy.loginViaApi(email, senha)
cy.obterToken(email, senha)
```

Uma pergunta para levar para casa

Abra os arquivos que você escreveu hoje e ontem. Conte quantas vezes `'[data-testid="entrar"]'` aparece.

Se o time de desenvolvimento renomear esse atributo amanhã, quantos arquivos você vai precisar abrir?

Não responda agora. Deixe a pergunta incomodar até amanhã de manhã.

Resumo do Dia 2

1. **Alias de requisição** transforma espera em fato: `cy.wait('@login')`, nunca `cy.wait(3000)`
2. **API é mais rápida e mais estável** que a tela — use para preparar estado
3. `failOnStatusCode: false` para testar o caminho que dá errado
4. `cy.intercept` **simula o impossível** — e às vezes encontra bug de verdade
5. **Login via API** como setup padrão: teste o login uma vez, não trinta
6. **Fixture +** `forEach` transforma cinco testes copiados em um teste e um JSON
7. **Comando customizado com valores padrão** funciona sem argumento e aceita sobrescrita

APÊNDICES

Consulta rápida

A parte da apostila que você vai usar depois do treinamento.

> DICA
> **Estes apêndices crescem com o curso.** Aqui está tudo o que foi visto até hoje. A versão completa vem na apostila do Dia 3.

A Comandos do Cypress por finalidade

Navegação

JAVASCRIPT

```
cy.visit('/login')          // abre uma página (relativa à baseUrl)
cy.reload()                 // recarrega
cy.go('back')               // volta
cy.go('forward')            // avança
cy.url()                    // a URL atual
cy.title()                  // o título da aba
```

Encontrar elementos

JAVASCRIPT

```
cy.get('[data-testid="email"]')    // por seletor CSS
cy.contains('Entrar')              // por texto
cy.contains('button', 'Entrar')    // por texto, dentro de um seletor
cy.get('form').find('input')       // dentro do elemento anterior
cy.get('li').first()               // primeiro
cy.get('li').last()                // último
cy.get('li').eq(2)                 // terceiro (conta do zero)
cy.get('td').parent()              // o pai
cy.get('td').parents('tr')         // o ancestral que casar com o seletor
cy.get('ul').children()            // os filhos diretos
```

Interagir

```javascript
.click()
.click({ force: true })       // força o clique — use com muita parcimônia
.dblclick()
.type('texto')
.type('texto', { log: false })      // não mostra o valor no log (senhas)
.type('{enter}')              // teclas especiais entre chaves
.clear()
.check()                      // checkbox ou radio
.uncheck()
.select('Opção')              // <select>
.trigger('mouseover')
.scrollIntoView()
```

Trabalhar com valores

```javascript
.invoke('val')                // chama um método do elemento
.invoke('text')
.its('response.statusCode')   // acessa uma propriedade
.then((valor) => { })         // recebe o valor no callback
cy.wrap(objeto)               // transforma um valor comum em algo "cypressável"
```

Rede

```javascript
cy.request('GET', url)                          // requisição direta, fora da tela
cy.intercept('GET', '**/rota*').as('apelido')     // observa
cy.intercept('GET', '**/rota*', { fixture: 'x.json' })   // simula com arquivo
cy.intercept('GET', '**/rota*', { statusCode: 500 })     // simula erro
cy.wait('@apelido')                             // espera acontecer
```

Arquivos e dados

```javascript
cy.fixture('usuarios')               // lê um arquivo de cypress/fixtures
cy.fixture('usuarios').as('usuarios') // e guarda com apelido
```

Debug

```javascript
cy.log('mensagem')
cy.pause()
.debug()
cy.screenshot('nome')
```

Linha de comando

```terminal
npm run cy:open                                  # escrever testes
npm run cy:run                                   # rodar tudo
npx cypress run --spec "caminho/do/teste.cy.js"  # rodar um só
npx cypress run --headed                         # ver o navegador
npx cypress run --browser chrome                 # escolher o navegador
npx cypress run --env chave=valor                # passar variável
```

B Tabela de asserções

Existência e visibilidade

```javascript
.should('exist')
.should('not.exist')
.should('be.visible')
.should('not.be.visible')
```

Conteúdo

```javascript
.should('have.text', 'exatamente este texto')
.should('contain', 'parte do texto')
.should('not.contain', 'este texto não')
.should('have.value', 'valor do campo')
.should('be.empty')
```

Quantidade

```javascript
.should('have.length', 3)
.should('have.length.greaterThan', 0)
.should('have.length.at.least', 1)
```

Estado

JAVASCRIPT

```
.should('be.checked')
.should('not.be.checked')
.should('be.disabled')
.should('be.enabled')
.should('have.focus')
```

Atributos e classes

JAVASCRIPT

```
.should('have.class', 'ativo')
.should('not.have.class', 'erro')
.should('have.attr', 'href', '/login')
.should('have.css', 'display', 'none')
```

Encadeando

JAVASCRIPT

```
cy.get('[data-testid="entrar"]')
  .should('be.visible')
  .and('contain', 'Entrar')      // .and é um .should encadeado
  .and('not.be.disabled')
```

`expect` — dentro de um `.then()`

JAVASCRIPT

```
cy.request('GET', url).then((res) => {
  expect(res.status).to.eq(200)
  expect(res.body).to.have.property('quantidade')
  expect(res.body.usuarios).to.be.an('array')
  expect(res.body.message).to.contain('sucesso')
  expect(res.body._id).to.not.be.empty
  expect(res.body).to.deep.include({ nome: 'Fulano' })
})
```

DICA

`should` ou `expect`? `should` tenta de novo até passar — use na tela. `expect` roda uma vez só — use dentro de `.then()`, quando o valor já chegou.

C Como ler uma falha: o método de 4 passos

1. Leia a primeira linha do erro. Ela diz o que o Cypress esperava e o que encontrou. Não pule para o código.

2. Olhe o screenshot da falha. A tela estava como você imaginava? Tinha um modal? Um carregando? A página abriu?

3. Volte um comando no Time Travel. Passe o mouse sobre o comando anterior. O estado antes da falha já estava errado? Se sim, o problema não é onde você achou.

4. Confira o seletor no DevTools.

```javascript
document.querySelectorAll('[data-testid="email"]')
```

Vazio? O seletor está errado ou o elemento ainda não existe.

D Mensagens de erro e o que significam

Mensagem	O que está acontecendo	O que fazer
`Timed out retrying... Expected to find element`	O seletor não encontrou nada no tempo	Confira o seletor no DevTools; veja se o elemento depende de uma requisição
`element is detached from the DOM`	A página se redesenhou entre achar e agir	Refaça o `cy.get()` mais perto da ação, ou espere a requisição terminar antes
`cy.click() failed because this element is being covered`	Tem algo por cima	Feche o modal/overlay. `{ force: true }` é último recurso, e esconde o problema
`cy.visit() failed trying to load`	URL errada ou aplicação fora do ar	Abra a URL no navegador. Se não abrir, rode `npm start` no `serverest-treinamento`
`Cypress.env() was removed in Cypress version 16.0.0`	O projeto está no Cypress 16, e o material é do 15	`npm install cypress@15.21.1 --save-dev`, ou migre para `Cypress.expose()` / `cy.env()` (bloco 2.5)
`Cypress could not verify that this server is running`	O `cypress run` conferiu o `baseUrl` antes de começar e não achou a aplicação	Mesma coisa: suba a loja (`npm start`) e rode de novo
`Cannot read properties of undefined`	Acesso a algo que não existe	Geralmente um `.then()` recebendo resposta vazia, ou fixture com estrutura diferente
`cy.type() can only accept a string or number`	Você passou `undefined` — uma variável que ninguém preencheu	Confira de onde vem o valor: fixture com a chave errada, ou variável que outro teste deveria ter preenchido (bloco 1.6)
Aviso `allowCypressEnv` no terminal	`Cypress.env()` depreciado no Cypress 15	Nada a fazer por ora; substitutos: `Cypress.expose()` e `cy.env()` (bloco 2.5)
`bad option: --smoke-test`	`ELECTRON_RUN_AS_NODE` no terminal	`unset ELECTRON_RUN_AS_NODE` (bash) ou `$env:ELECTRON_RUN_AS_NODE=$null` (PowerShell)

Dados, API e rede (Dia 2)

Mensagem	O que está acontecendo	O que fazer
`cy.type() cannot accept an empty string`	Você passou `''`	Não digite nada: `if (valor) cy.get(...).type(valor)` — como no bloco 2.5
`Cannot call cy.fixture() outside a running test`	Comando do Cypress fora de um `it` ou hook	Para gerar testes em loop, use `require` em vez de `cy.fixture()`
`cy.wait() timed out waiting for the 1st request`	O intercept não casou com a rota	Foi registrado depois do `visit`? O padrão casa com a URL real (confira na aba Network)?

E Seletores do ServeRest

> Conferidos contra a aplicação real. Se algo aqui não funcionar, a aplicação mudou — ela é de estudo e muda sem avisar.

`/login`

Elemento	Seletor
E-mail	`[data-testid="email"]`
Senha	`[data-testid="senha"]`
Entrar	`[data-testid="entrar"]`
Link cadastrar	`[data-testid="cadastrar"]`
Alerta	`.alert`

`/cadastrarusuarios`

Elemento	Seletor
Nome	`[data-testid="nome"]`
E-mail	`[data-testid="email"]`
Senha	`[data-testid="password"]` ⚠️
Checkbox admin	`[data-testid="checkbox"]`
Cadastrar	`[data-testid="cadastrar"]`

`/admin/home`

Elemento	Seletor
Cadastrar usuários	`[data-testid="cadastrarUsuarios"]`
Cadastrar produtos	`[data-testid="cadastrarProdutos"]`
Listar produtos	`[data-testid="listarProdutos"]`
Listar usuários	`[data-testid="listarUsuarios"]`
Logout	`[data-testid="logout"]`

`/admin/cadastrarprodutos`

Elemento	Seletor
Nome	`[data-testid="nome"]`
Preço	`[data-testid="preco"]`
Descrição	`[data-testid="descricao"]`
Quantidade	`[data-testid="quantity"]` ⚠️ em inglês
Imagem	`[data-testid="imagem"]`
Cadastrar	`[data-testid="cadastarProdutos"]` ⚠️ typo da aplicação

Listagens do admin

Não têm `data-testid`. Ancore no texto:

JAVASCRIPT

```
cy.contains('table.table-striped tbody tr', 'Nome do Produto')
  .contains('button', 'Excluir')
  .click()
```

Títulos das telas

Rota	`<h1>`
`/admin/home`	`Bem Vindo <nome>`
`/home` (loja)	`Serverest Store`
`/admin/listarprodutos`	`Lista dos Produtos`
`/admin/listarusuarios`	`Lista dos usuários`

As três armadilhas

1. `senha` **é só do login.** No cadastro é `password`.
2. **O botão de cadastrar produto tem typo:** `cadastarProdutos`, sem o segundo "r".
3. `Serverest Store` **não é a home do admin.** É a loja do usuário comum.

F Comandos customizados do projeto

Interface

JAVASCRIPT

```javascript
cy.login(email, senha)
// Login pela tela. Existe para ser TESTADO, não para ser usado como setup.

cy.loginViaApi(email, senha)
// Login pela porta dos fundos. É o setup padrão.
// Sem argumentos, usa admin.treinamento@qa.com / treino123.

cy.get('h1').validarTexto('Bem Vindo')
// Comando filho: valida o texto do elemento anterior.

cy.emailUnico('prefixo')
// Gera um e-mail único.
```

API

JAVASCRIPT

```javascript
cy.obterToken(email, senha)
// Devolve o token no formato do header. Sem argumentos, usa o admin padrão.

cy.criarUsuarioViaApi({ nome, email, password, administrador })
// Todos os campos são opcionais. Devolve o usuário criado, com _id.

cy.cadastrarProdutoViaApi({ nome, preco, descricao, quantidade })
// Resolve a autenticação por dentro. Devolve o produto criado, com _id.

cy.excluirProdutoViaApi(id)
cy.excluirUsuarioViaApi(id)
```

Exemplos de uso

```javascript
// Criar um usuário e entrar com ele
cy.criarUsuarioViaApi().then((usuario) => {
  cy.loginViaApi(usuario.email, usuario.password)
  cy.visit('/admin/home')
  cy.contains('h1', usuario.nome).should('be.visible')
})

// Preparar um produto e validar na tela
cy.loginViaApi()
cy.cadastrarProdutoViaApi({ nome: `Produto ${Date.now()}` }).then((produto) => {
  cy.visit('/admin/listarprodutos')
  cy.contains('td', produto.nome).should('be.visible')
})
```

G Endpoints da API do ServeRest

Base: `http://localhost:3000` (a sua cópia, do `serverest-treinamento`). A pública, com as mesmas rotas, é `https://serverest.dev`.

Verbo	Rota	Autenticação	Uso
POST	`/login`	—	Autenticar, obter token
GET	`/usuarios`	—	Listar (aceita `?email=`, `?nome=`)
GET	`/usuarios/{id}`	—	Buscar um
POST	`/usuarios`	—	Criar
DELETE	`/usuarios/{id}`	—	Excluir
GET	`/produtos`	—	Listar
GET	`/produtos/{id}`	—	Buscar um
POST	`/produtos`	✅ Token	Criar
DELETE	`/produtos/{id}`	✅ Token	Excluir
POST	`/carrinhos`	✅ Token	Criar carrinho

Respostas que você vai encontrar

Situação	Status	Corpo
Login OK	200	`{ message: "Login realizado com sucesso", authorization: "Bearer ..." }`
Login inválido	401	`{ message: "Email e/ou senha inválidos" }`
Login sem o campo e-mail	400	`{ email: "email é obrigatório" }`
Cadastro OK	201	`{ message: "Cadastro realizado com sucesso", _id: "..." }`
E-mail duplicado	400	`{ message: "Este email já está sendo usado" }`
Produto sem token	401	`{ message: "Token de acesso ausente, inválido..." }`
Produto com nome repetido	400	`{ message: "Já existe produto com esse nome" }`
Exclusão OK	200	`{ message: "Registro excluído com sucesso" }`

Autenticação

JAVASCRIPT

```javascript
cy.request('POST', `${apiUrl}/login`, {
  email: 'admin.treinamento@qa.com',
  password: 'treino123'
}).then((res) => {
  const token = res.body.authorization   // já vem com "Bearer "

  cy.request({
    method: 'POST',
    url: `${apiUrl}/produtos`,
    headers: { authorization: token },
    body: { nome: 'x', preco: 1, descricao: 'y', quantidade: 1 }
  })
})
```

`localStorage` do front

Chave	Para quê
`serverest/userToken`	O que o guarda de rota confere
`serverest/userNome`	O que a home do admin imprime
`serverest/userEmail`	Impede excluir o próprio usuário

CRÍTICO

Faça um `cy.visit` **antes** de escrever no `localStorage`. É ele que estabelece a origem.